

LAB2 Report: ROS Installation and TF-Based Two-Drone Trajectory Analysis

Haoran Xiao
Student No.: 24020036066

Abstract—This report documents the completion of a ROS laboratory on installation, workspace configuration, node and topic inspection, TF broadcasting, TF lookup, visualization, and rigid-transformation analysis. The course material specifies Ubuntu 20.04 with ROS1 Noetic, while the available Lab1 virtual machine was Ubuntu 22.04.5 LTS with no native ROS installation. To preserve the required ROS1 environment without replacing the virtual machine, Docker was used to run the official OSRF ROS Noetic desktop image. The ROS1-compatible Lab2 package was selected at commit 609f31f after the current repository tip was correctly identified as a ROS2 `ament_cmake` package. The workspace compiled successfully, the static and moving two-drone scenes ran in rViz, the node/topic graph was inspected, and the relative trajectory was visualized in the AV1 frame. The measured mathematical result is that the relative trajectory lies on $z - 2y = 1$ and is an ellipse with semi-axis lengths $1/2$ and $\sqrt{5}/2$. The evidence supports successful software-in-the-loop visualization and compilation; it does not establish physical-robot execution.

I. INTRODUCTION AND OBJECTIVES

Robot Operating System (ROS) provides a communication and tooling layer for robotics applications. In a visual-navigation system, a single geometric object is commonly represented in several coordinate frames, such as a world frame, a vehicle body frame, and sensor frames. The ability to publish, inspect, and query these frame relationships is therefore a prerequisite for later navigation, estimation, and control laboratories.

This laboratory had four connected objectives. First, it required a working ROS environment and a catkin workspace. Second, it required inspection of ROS nodes, topics, launch files, and the rViz visualization pipeline. Third, it required implementation of two ROS1 nodes: one to publish the prescribed motion of two aerial vehicles through TF, and one to query a relative transform and draw the resulting trajectory. Finally, it required mathematical derivations using homogeneous transformations, rotation matrices, and quaternion identities.

The experiment was designed to connect implementation and theory. The code makes the two trajectories observable, while the coordinate-frame analysis explains why the trajectory of AV2 relative to AV1 is an ellipse on a slanted plane. This combination is relevant to later robotics projects because a program may compile while still using the wrong frame direction, and a visually plausible plot may still represent the wrong transform.

The expected outcome was a reproducible ROS1 workspace containing the Lab2 package, a successful catkin build, a static and dynamic rViz demonstration, a node/topic

TABLE I
VERIFIED EXPERIMENTAL ENVIRONMENT.

Item	Verified value
Host OS	Ubuntu 22.04.5 LTS in the Lab1 virtual machine
Container image	<code>osrf/ros:noetic-desktop-full</code>
ROS distribution	ROS1 Noetic; <code>rosversion -d</code> returned <code>noetic</code>
Workspace	<code>/root/vnav_ws</code> inside the container
Build system	<code>catkin_tools</code> ; linked devel space
Visualization	rViz and <code>rqt_graph</code> through the container display
Source version	MIT-SPARK/VNAV-labs commit 609f31f
Hardware	No physical robot or UAV was used; simulation/visualization only

graph, a relative-frame visualization, and a report supported by traceable evidence.

II. EXPERIMENTAL ENVIRONMENT

A. Required and Actual Environments

The supplied Lab2 handout specifies Ubuntu 20.04 with ROS1 Noetic. The available Lab1 virtual machine was instead Ubuntu 22.04.5 LTS, and the initial checks showed that neither `/opt/ros` nor the ROS commands were present. Installing only `python3-rospkg` would not have installed a complete ROS distribution, so that route was not used.

The actual solution kept the existing virtual machine as the host and ran Ubuntu 20.04 with ROS Noetic inside the OSRF Docker image `osrf/ros:noetic-desktop-full`. This separated the host operating-system version from the ROS distribution required by the experiment. The container was launched with host networking, the X11 socket, the display variable, and a persistent workspace mount so that ROS nodes and rViz could run together.

The final report, source files, evidence, and reproducibility materials are archived in the public repository <https://github.com/xhr-CHN/Lab2-ROS-Noetic-ICRA-Report>.

Figure 1 records the successful Docker smoke test. Figure 2 records the Noetic image download, and Fig. 3 records the Noetic version and catkin workspace initialization. These figures establish the environment boundary used by the rest of the report.

III. EXPERIMENTAL PLAN AND PROCESS

The experiment was executed as a sequence of independently checkable stages. Each stage had a distinct purpose

```

Setting up containerd (2.2.1-0ubuntu1-22.04.2) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service
→ /lib/systemd/system/containerd.service.
Setting up ubuntu-fan (0.12.16) ...
Created symlink /etc/systemd/system/multi-user.target.wants/ubuntu-fan.service
→ /lib/systemd/system/ubuntu-fan.service.
Setting up docker.io (29.1.3-0ubuntu3-22.04.2) ...
Adding group 'docker' (GID 137) ...
Done.
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /l
ib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /lib/s
ystemd/system/docker.socket.
Processing triggers for man-db (2.10.2-1) ...
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Fig. 1. Docker smoke test returning “Hello from Docker!” on the Ubuntu 22.04 host.

```

xhr@xhr-virtual-machine:~$ docker pull osrf/ros:noetic-desktop-full
noetic-desktop-full: Pulling from osrf/ros
4f15d5036206: Pull complete
13b7e930469f: Pull complete
0da4792f7f41: Pull complete
d20358ca2eb5: Pull complete
5dce350d6ea4: Pull complete
60ca0507171d: Pull complete
59cc10980325: Pull complete
6932d16c2645: Pull complete
342ecf093a91: Pull complete
6c240714a1b7: Pull complete
ff0451913b17: Pull complete
3d5c979472de: Pull complete
1b468568b3ae: Pull complete
Digest: sha256:7dbfb9576d8e6d226c31e06129a82aaab8702695f38eca2116918cb9b9308797
Status: Downloaded newer image for osrf/ros:noetic-desktop-full
docker.io/osrf/ros:noetic-desktop-full

```

Fig. 2. Successful pull of the OSRF ROS Noetic desktop image.

and a visible success condition.

A. Stage 1: Docker access and ROS image preparation

The first input was the existing Ubuntu 22.04 virtual machine. Docker was installed and the user was added to the Docker group. The hello-world image verified communication among the Docker client, daemon, registry, and container runtime. The user-group check was then repeated after re-login; `id -nG` included `docker` and `docker ps` no longer returned a permission error. Figure 4 is the corresponding access evidence. The Noetic desktop image was pulled only after this permission check succeeded.

B. Stage 2: Workspace and source preparation

Inside the container, the ROS environment was sourced and the workspace was initialized with `catkin init`. The package was cloned from MIT-SPARK/VNAV-labs and copied into `/root/vnav_ws/src`. Figure 5 shows the package structure, including `CMakeLists.txt`, `package.xml`, launch files, configuration files, meshes, and source code.

```

root@xhr-virtual-machine:~# source /opt/ros/noetic/setup.bash
root@xhr-virtual-machine:~# rosversion -d
noetic
root@xhr-virtual-machine:~# cd /root/vnav_ws
root@xhr-virtual-machine:~/vnav_ws# catkin init
Initializing catkin workspace in '/root/vnav_ws'.
-----
Profile: default
Extending: [env] /opt/ros/noetic
Workspace: /root/vnav_ws
-----
Build Space: [missing] /root/vnav_ws/build
Devel Space: [missing] /root/vnav_ws/devel
Install Space: [unused] /root/vnav_ws/install
Log Space: [missing] /root/vnav_ws/logs
Source Space: [exists] /root/vnav_ws/src
DESTDIR: [unused] None
-----
Devel Space Layout: linked
Install Space Layout: None
-----
Additional CMake Args: None
Additional Make Args: None
Additional catkin Make Args: None
Internal Make Job Server: True
Cache Job Environments: False
-----
Buildlisted Packages: None
Skiplisted Packages: None
-----
Workspace configuration appears valid.
-----
root@xhr-virtual-machine:~/vnav_ws#

```

Fig. 3. ROS Noetic version check and valid catkin workspace initialization.

```

xhr@xhr-virtual-machine:~$ id -nG
docker ps
xhr adm cdrom sudo dip plugdev lpadmin lxd sambashare docker
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

```

Fig. 4. Non-root Docker access after the user-group update.

The first shallow clone used the current repository tip. Catkin reported that the package had an unsupported `ament_cmake` build type, so it skipped the package rather than compiling it. This was a diagnostic result, not a successful build, and is retained in Fig. 6. The full Git history was then fetched and the package history was inspected. Commits `2f199b4`, `04e9eba`, and `609f31f` used catkin, whereas `ed9d62f` used ROS2 `ament_cmake`. The check-out was changed to the latest ROS1-compatible commit, `609f31f`, and the ROS2 package directory was moved outside the source space as a recoverable backup. The resulting ROS1 package compiled successfully, as shown in Fig. 7.

C. Stage 3: Static ROS graph and visualization

The static scenario was launched with:

```

source /opt/ros/noetic/setup.bash
source /root/vnav_ws/devel/setup.bash
roslaunch two_drones_pkg two_drones.launch static:=True

```

The purpose of this stage was to validate the launch file, the package resources, the mesh markers, and the basic TF graph before introducing time-varying motion. The two drones were visible in rViz with `world` as the fixed frame, and the global status was OK, as shown in Fig. 8.

The node and topic checks were performed in a second container shell. The recorded nodes included `/av1broadcaster`, `/av2broadcaster`, `/plots_publisher_node`, `/rosout`, and `/rviz`. The recorded topics included `/tf`, `/tf_static`, `/visuals`, and standard rViz topics. Figure 9 shows the terminal output, while Fig. 10 shows the graph-level relationship between the broadcasters and the plotting node.

```

root@xhr-virtual-machine:~/vnav_ws# cd /root
root@xhr-virtual-machine:~# git clone --depth 1 https://github.com/MIT-SPARK/VN
AV-labs.git
Cloning into 'VNAV-Labs'...
remote: Enumerating objects: 225, done.
remote: Counting objects: 100% (225/225), done.
remote: Compressing objects: 100% (181/181), done.
remote: Total 225 (delta 38), reused 160 (delta 24), pack-reused 0 (from 0)
Receiving objects: 100% (225/225), 527.16 KiB | 1.71 MiB/s, done.
Resolving deltas: 100% (38/38), done.
root@xhr-virtual-machine:~# cp -a /root/VNAV-Labs/lab2/two_drones_pkg /root/vna
v_ws/src/
root@xhr-virtual-machine:~# ls -la /root/vnav_ws/src/two_drones_pkg
total 32
drwxr-xr-x 6 root root 4096 Sep 20 09:19 .
drwxrwxr-x 3 1000 1000 4096 Sep 20 09:19 ..
-rw-r--r-- 1 root root 1443 Sep 20 09:19 CMakeLists.txt
drwxr-xr-x 2 root root 4096 Sep 20 09:19 config
drwxr-xr-x 2 root root 4096 Sep 20 09:19 launch
drwxr-xr-x 2 root root 4096 Sep 20 09:19 mesh
-rw-r--r-- 1 root root 335 Sep 20 09:19 package.xml
drwxr-xr-x 2 root root 4096 Sep 20 09:19 src

```

Fig. 5. The Lab2 package copied into the catkin source space.

```

Profile: default
Extending: [env] /opt/ros/noetic
Workspace: /root/vnav_ws

Build Space: [exists] /root/vnav_ws/build
Devel Space: [exists] /root/vnav_ws/devel
Install Space: [unused] /root/vnav_ws/install
Log Space: [missing] /root/vnav_ws/logs
Source Space: [exists] /root/vnav_ws/src
DESTDIR: [unused] None

Devel Space Layout: Linked
Install Space Layout: None

Additional CMake Args: None
Additional Make Args: None
Additional catkin Make Args: None
Internal Make Job Server: True
Cache Job Environments: False

Buildlisted Packages: None
Skiplist Packages: None

Workspace configuration appears valid.

NOTE: Forcing CMake to run for each package.

[build] Found 1 packages in 0.0 seconds.
[build] Updating package table.
[build] Warning: Skipping package 'two_drones_pkg' because it has an unsupporte
d package build type: 'ament_cmake'
[build] Note: Available build types:
[build] - 'catkin'
[build] - 'cmake'
Starting >>> catkin_tools_prebuild
<<< catkin_tools_prebuild [ 1.2 seconds ]
[build] Summary: All 1 packages succeeded!
[build] Ignored: 1 packages were skipped or are skiplisted.
[build] Runtime: 1.2 seconds total.

```

Fig. 6. Diagnostic first build: the current ROS2 package was skipped by catkin because of ament_cmake.

D. Stage 4: Time-varying TF publication

The first implementation task was to complete `frames_publisher_node.cpp`. The node runs a 50 Hz timer. At each callback it computes elapsed time, constructs two `geometry_msgs/TransformStamped` messages, sets the frame names and translations, computes AV1's yaw quaternion, and broadcasts both transforms.

The input motion specification was:

$$o_1^w(t) = [\cos t, \sin t, 0]^T, \quad (1)$$

$$o_2^w(t) = [\sin t, 0, \cos(2t)]^T. \quad (2)$$

The AV1 orientation uses `setRPY(0,0,t)`, which makes its y -axis tangent to its circular trajectory while keeping its z -axis parallel to the world z -axis. The successful build of this implementation is shown in Fig. 11.

```

Profile: default
Extending: [cached] /opt/ros/noetic
Workspace: /root/vnav_ws

Build Space: [exists] /root/vnav_ws/build
Devel Space: [exists] /root/vnav_ws/devel
Install Space: [unused] /root/vnav_ws/install
Log Space: [exists] /root/vnav_ws/logs
Source Space: [exists] /root/vnav_ws/src
DESTDIR: [unused] None

Devel Space Layout: Linked
Install Space Layout: None

Additional CMake Args: None
Additional Make Args: None
Additional catkin Make Args: None
Internal Make Job Server: True
Cache Job Environments: False

Buildlisted Packages: None
Skiplist Packages: None

Workspace configuration appears valid.

NOTE: Forcing CMake to run for each package.

[build] Found 1 packages in 0.0 seconds.
[build] Package table is up to date.
Starting >>> two_drones_pkg
<<< two_drones_pkg [ 3.8 seconds ]
[build] Summary: All 1 packages succeeded!
[build] Ignored: 0 packages were skipped or are skiplisted.
[build] Runtime: 3.8 seconds total.
[build] Note: Workspace packages have changed, please re-source setup files to
use them.

```

Fig. 7. Successful catkin build after switching to the ROS1-compatible commit 609f31f.

E. Stage 5: Dynamic visualization and relative transform lookup

The static launch was stopped and the non-static launch was started without the `static:=True` argument. Two screenshots taken at different times show that the drones changed position, as illustrated in Figs. 12 and 13. This is runtime evidence for the simulated visualization, not evidence of a physical UAV.

For Deliverable 3, the plotting node was completed with:

```

transform = parent->tf_buffer.lookupTransform(
    ref_frame, dest_frame, ros::Time(0));

```

This call requests the latest available transform of the destination frame expressed in the reference frame. After recompilation, the plotting node generated the relative trajectory, and its successful build is recorded in Fig. 14. Changing rViz's fixed frame from `world` to `av1` produced the world-frame circle, the AV2 parabola, and the dashed relative trajectory in Fig. 15.

F. Stage 6: turtlesim and Twist-based motion control

The remaining course tasks were completed with the ROS Noetic `turtlesim` mobile-robot simulator. This is consistent with the handout's wording that the control exercise may use an unmanned system or mobile robot in simulation. First, `turtlesim_node` and `turtle_teleop_key` were launched to verify the standard ROS node and topic path, as shown in Fig. 16. The subsequent visible motion and command completion are demonstrated separately by the custom controller in Fig. 17.

The second control test used a custom Python node named `turtle_motion_controller`. The node published

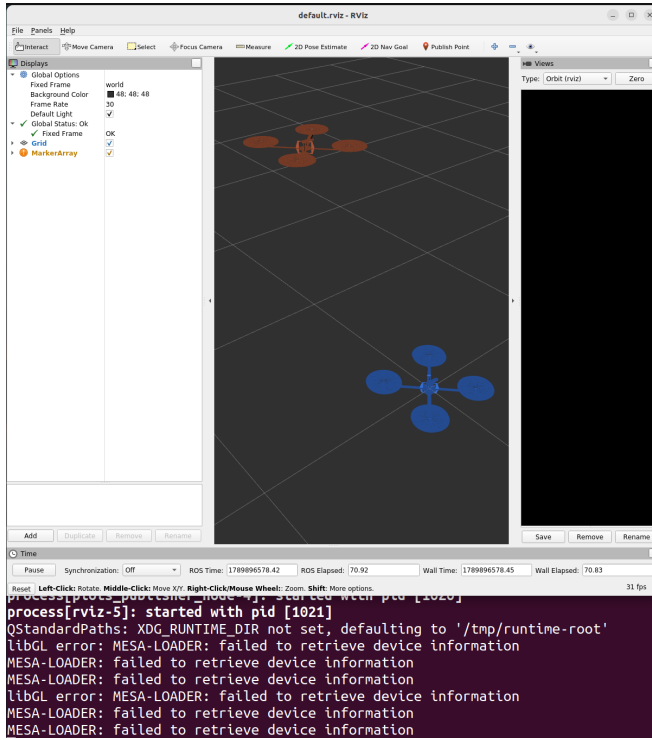


Fig. 8. Static two-drone scene in rViz with the world frame fixed.

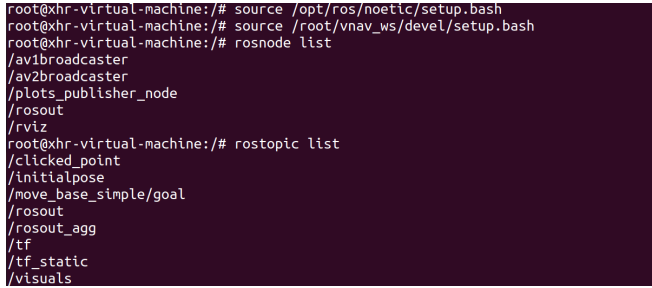


Fig. 9. Recorded ROS node and topic lists in the static scenario.

geometry_msgs/Twist messages to /turtle1/cmd_vel at 10 Hz. It commanded a forward velocity of 1.0 and an angular velocity of 0.6 for 12 s, then published a zero command to stop the turtle. The terminal recorded both the start and completion messages while the simulator displayed the resulting circular trajectory in Fig. 17.

G. Notion-page verification extensions

The Notion lab page also specifies several ROS inspection and TF tools. After correcting the container DNS configuration, rosdep update completed and updated the local cache, as shown in Fig. 18. The TurtleSim velocity channel was then observed with rostopic echo, and the node/topic relationship was inspected with rqt_graph in Figs. 19 and 20.

The standard turtle_tf demonstration was launched after installing the missing python-is-python3 compatibility command. The two turtle broadcasters, listener, and simulator ran together in Fig. 21. The TF tree, a numerical transform query, and an rViz TF display were then verified in

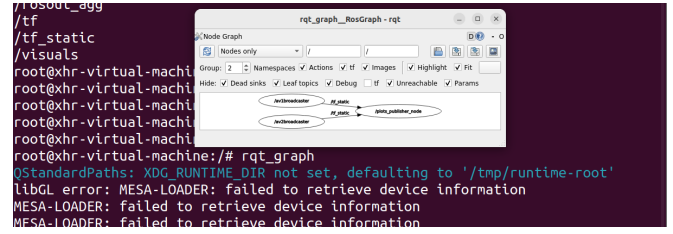


Fig. 10. rqt_graph view showing the two broadcasters and the plotting node.

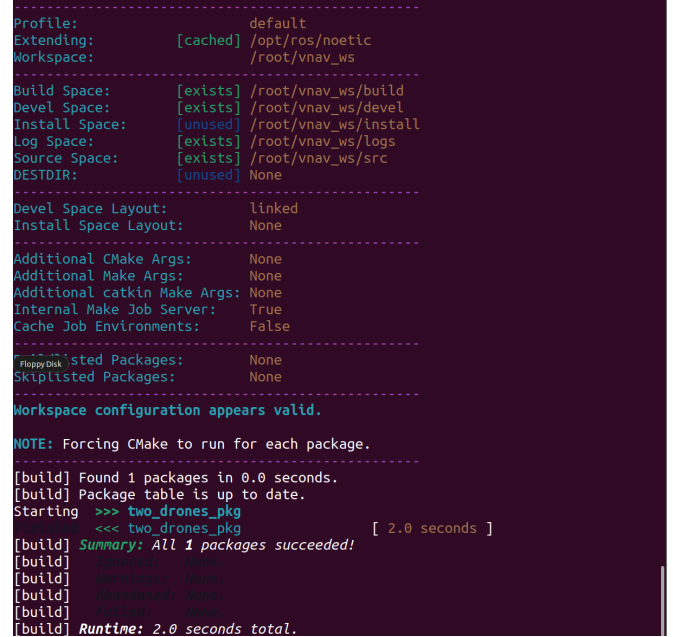


Fig. 11. Successful build after implementing the time-varying TF broadcaster.

Figs. 22, 23, and 24. These checks complete the previously missing Notion-page tool exercises. The rosdep command was executed as the container root user; its root warning is recorded, while the actual source-cache update succeeded.

IV. IMPLEMENTATION AND EXPERIMENTAL METHOD

A. Package structure and data flow

The package contains a catkin manifest, a CMake build file, launch and configuration directories, meshes, and C++ source files. The launch file starts the two broadcasters, the plotting node, and rViz. The broadcasters publish TF transforms; the plotting node queries TF and converts the resulting translation into points stored in a bounded trajectory buffer; rViz subscribes to the visual marker output and displays the vehicle meshes and trails.

The data flow is therefore:

time → TransformStamped → TF → lookupTransform → marker trajectory → rViz.

B. Transform publisher

The timer period is 0.02 s, corresponding to 50 Hz. The publisher uses a common timestamp for the two transforms

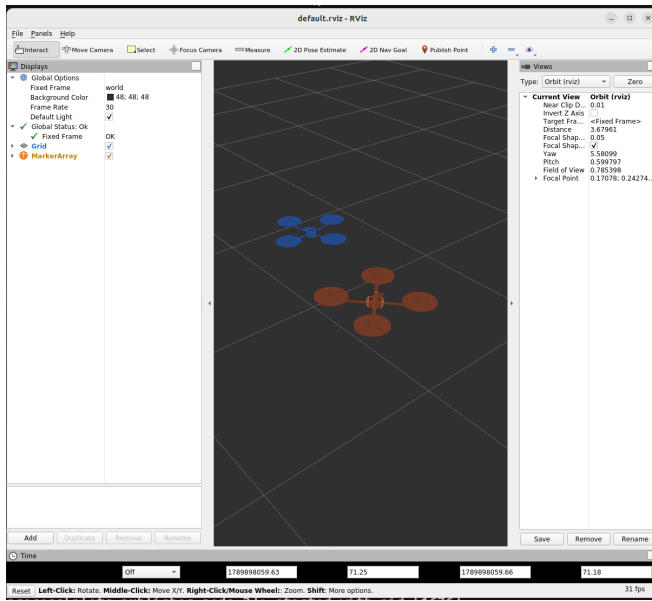


Fig. 12. First recorded view of the moving two-drone scene.

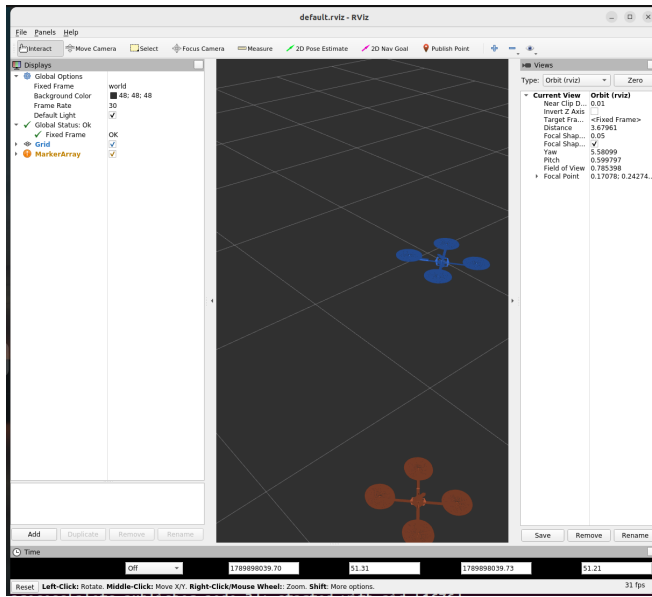


Fig. 13. Second recorded view of the moving two-drone scene at a later time.

in each callback. AV1 has parent frame world, child frame av1, position $[\cos t, \sin t, 0]^T$, and yaw t . AV2 has parent frame world, child frame av2, position $[\sin t, 0, \cos(2t)]^T$, and an identity orientation because its orientation is irrelevant to the requested positional analysis.

The key design choice is the AV1 orientation. With yaw t , its body y -axis is $[-\sin t, \cos t, 0]^T$, which is tangent to the circle. This agrees with the problem statement even though the conventional forward axis of a vehicle is often chosen as x .

```

Profile: default
Extending: [cached] /opt/ros/noetic
Workspace: /root/vnav_ws

Build Space: [exists] /root/vnav_ws/build
Devel Space: [exists] /root/vnav_ws/devel
Install Space: [unused] /root/vnav_ws/install
Log Space: [exists] /root/vnav_ws/logs
Source Space: [exists] /root/vnav_ws/src
DESTDIR: [unused] None

Devel Space Layout: Linked
Install Space Layout: None

Additional CMake Args: None
Additional Make Args: None
Additional catkin Make Args: None
Internal Make Job Server: True
Cache Job Environments: False

Buildlisted Packages: None
Skiplist Packages: None

Workspace configuration appears valid.

NOTE: Forcing CMake to run for each package.

[build] Found 1 packages in 0.0 seconds.
[build] Package table is up to date.
Starting >>> two_drones_pkg [ 3.0 seconds ]
[build] Summary: All 1 packages succeeded!
[build]
[build]
[build]
[build]
[build]
[build] Runtime: 3.0 seconds total.

```

Fig. 14. Successful build after implementing the relative transform lookup.

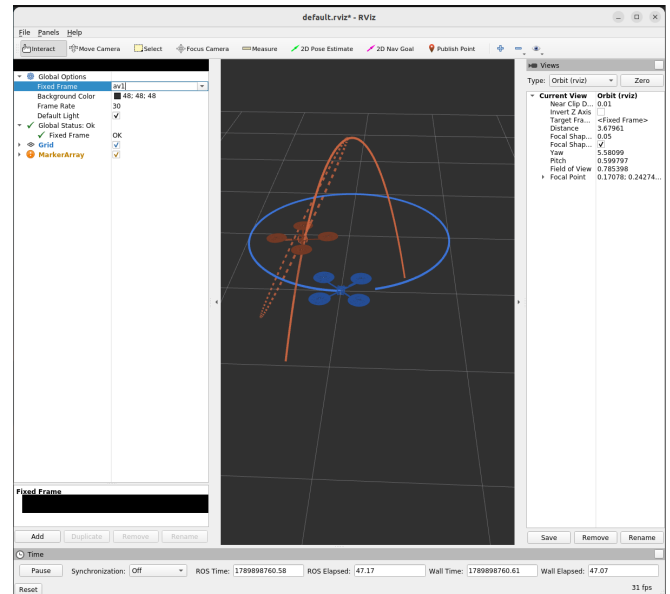


Fig. 15. rViz with AV1 as the fixed frame, showing the absolute and relative trajectories.

C. Relative transform lookup

The lookup uses the TF2 convention `lookupTransform(target, source, time)`. Passing `refframe` as the target and `destframe` as the source returns the destination pose expressed in the reference frame. The call is inside a TF exception handler, so a temporary absence of a transform is reported rather than silently converted into a false trajectory point.

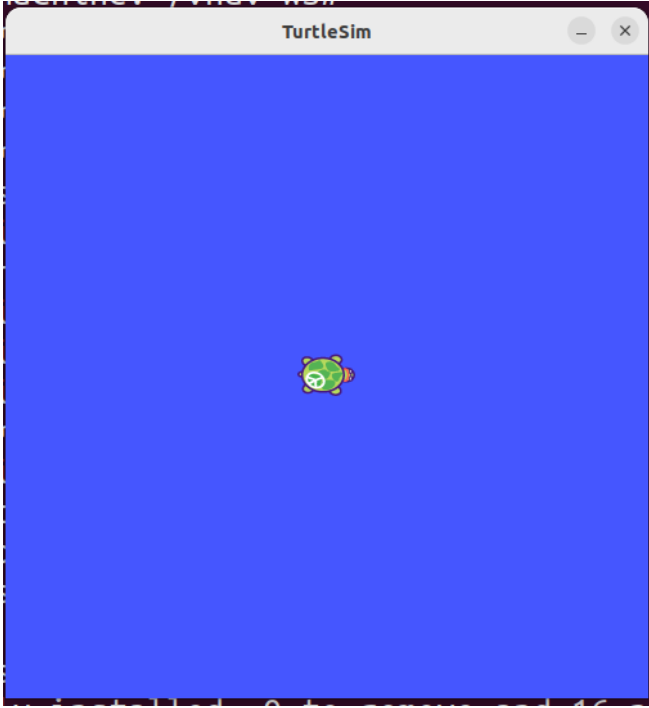


Fig. 16. Turtlesim and the keyboard teleoperation node launched in the ROS Noetic container.

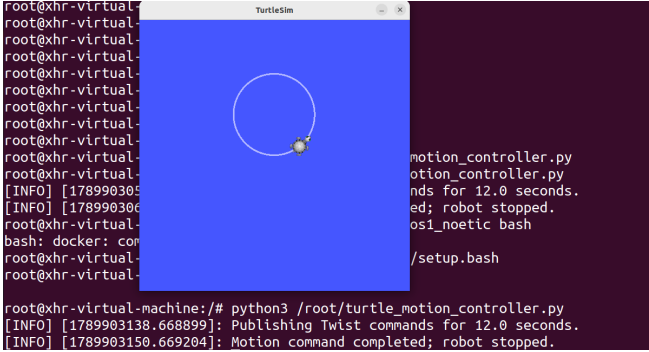


Fig. 17. Custom Twist controller producing a circular turtlesim trajectory and reporting command completion.

D. Mathematical derivation

The world-frame AV2 trajectory satisfies $y = 0$, $x = \sin t$, and $z = \cos(2t)$. Therefore:

$$z = 1 - 2x^2, \quad (3)$$

which proves that it is a parabola in the world x - z plane.

The AV1-to-world rotation is $R_1^w = R_z(t)$, so the world-to-AV1 rotation is:

$$R_w^1 = R_z(-t) = \begin{bmatrix} \cos t & \sin t & 0 \\ -\sin t & \cos t & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (4)$$

Using $c = \cos t$ and $s = \sin t$, the relative position is:

$$o_2^1(t) = R_w^1(o_2^w - o_1^w) = \begin{bmatrix} cs - 1 \\ -s^2 \\ \cos(2t) \end{bmatrix}. \quad (5)$$

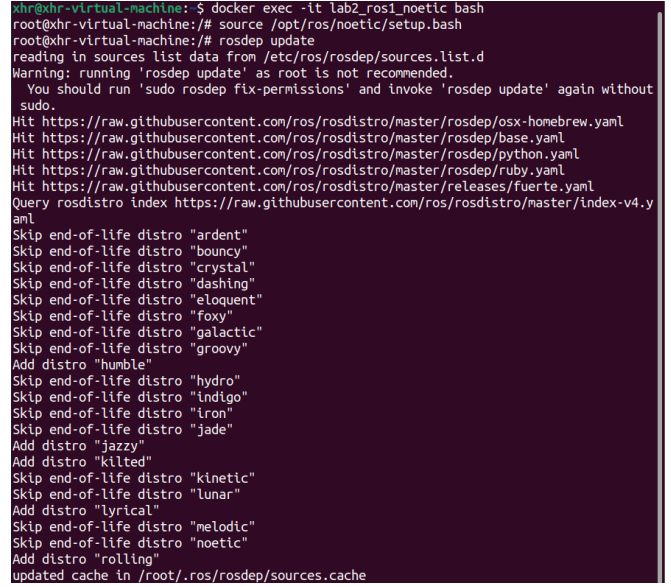


Fig. 18. Successful rosdep source-cache update after repairing container DNS resolution.

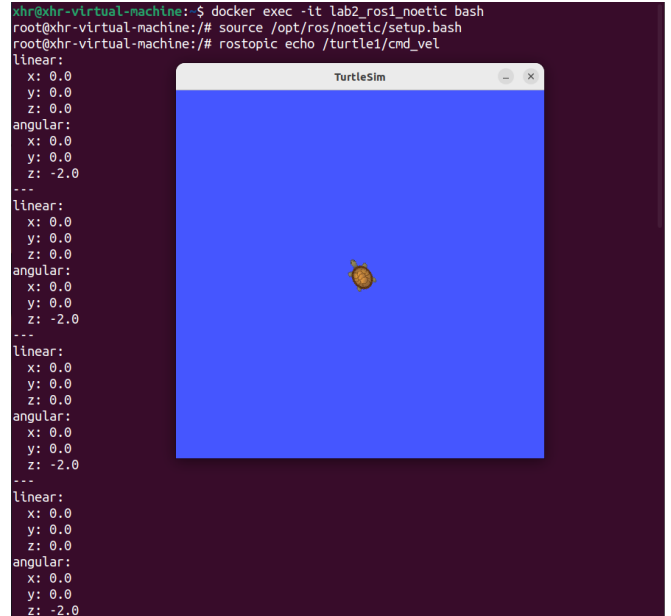


Fig. 19. Observed `/turtle1/cmd_vel` messages from the teleoperation node.

Because $z = 1 - 2s^2 = 1 + 2y$, the trajectory lies on:

$$z - 2y = 1. \quad (6)$$

The center of the curve is $p = [-1, -1/2, 0]^T$. An orthonormal basis for the plane is:

$$e_x = [1, 0, 0]^T, \quad e_y = \frac{1}{\sqrt{5}}[0, 1, 2]^T. \quad (7)$$

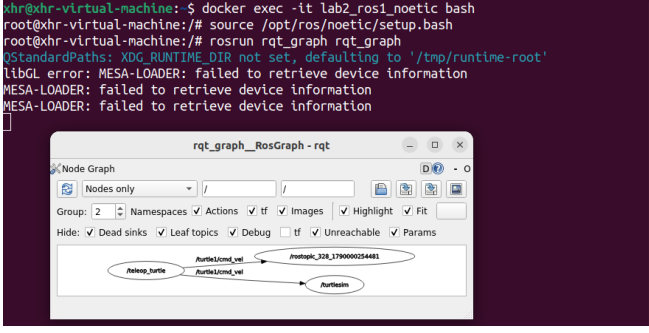


Fig. 20. rqt_graph for the TurtleSim teleoperation communication path.

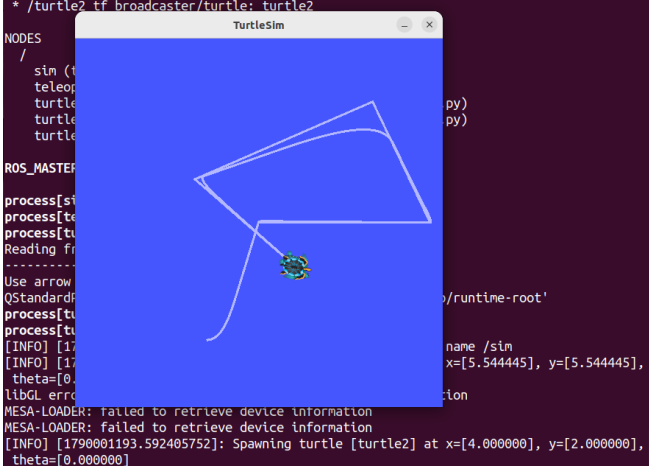


Fig. 21. The standard turtle TF demonstration with the simulator and TF nodes running.

The centered coordinates are:

$$x_p = \frac{1}{2} \sin(2t), \quad (8)$$

$$y_p = \frac{\sqrt{5}}{2} \cos(2t). \quad (9)$$

Consequently:

$$\frac{x_p^2}{(1/2)^2} + \frac{y_p^2}{(\sqrt{5}/2)^2} = 1. \quad (10)$$

The semi-axis lengths are therefore $1/2$ and $\sqrt{5}/2$, with the longer axis aligned with e_y .

E. Quaternion identities

For a unit quaternion q , left multiplication by q and right multiplication by q preserve the Euclidean norm because quaternion norms multiply. Therefore the matrices $\Omega_1(q)$ and $\Omega_2(q)$ are orthogonal. Since $q^{-1} = [-q_1, -q_2, -q_3, q_4]^T$, their transposes represent multiplication by the inverse and satisfy:

$$\Omega_1(q)^T q = \Omega_2(q)^T q = [0, 0, 0, 1]^T. \quad (11)$$

Finally, left and right quaternion multiplication commute by associativity:

$$\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x). \quad (12)$$

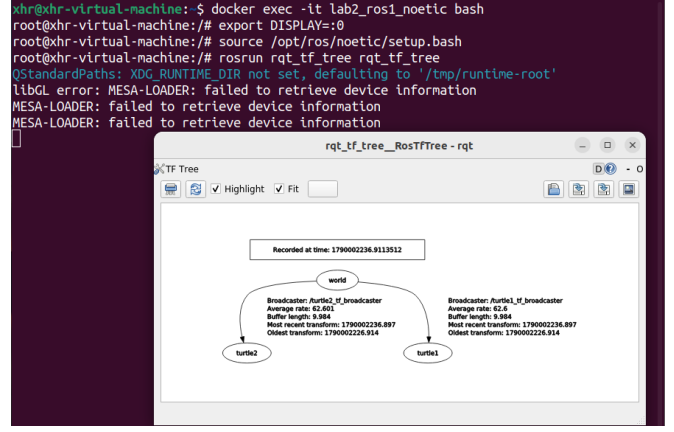


Fig. 22. rqt_tf_tree showing the world, turtle1, and turtle2 frame hierarchy.

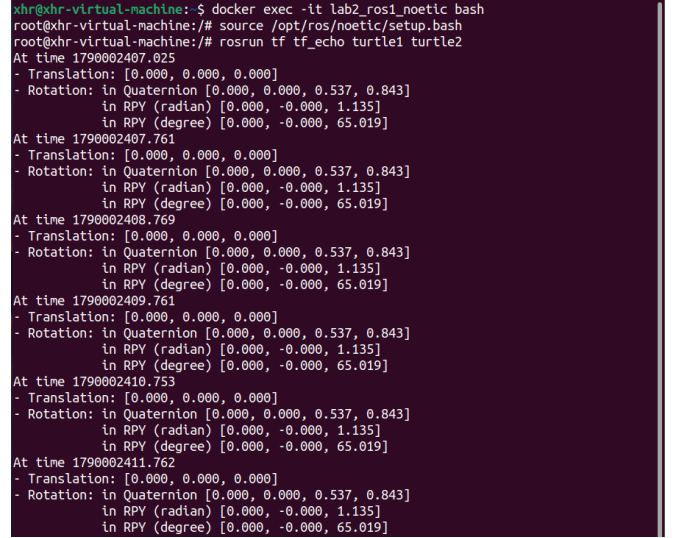


Fig. 23. Numerical TF query from turtle1 to turtle2, including quaternion and RPY output.

The transpose identity follows because $\Omega_2(y)^T = \Omega_2(\bar{y})$, where $\bar{y}$ is the quaternion conjugate.

F. Optional Deliverable 6: intrinsic and extrinsic rotations

Let R_0 , R_1 , and R_2 denote the rotations listed in the optional exercise. For the extrinsic interpretation, all rotations are about fixed world axes. Applying R_0 , then R_1 , then R_2 gives

$$R_{\text{ext}} = R_2 R_1 R_0. \quad (13)$$

For the intrinsic interpretation, the order is reversed. Starting with the body frame, apply the local rotations R_2 , then R_1 , then R_0 . A local rotation post-multiplies the current orientation, so

$$R_{\text{int}} = I R_2 R_1 R_0 = R_2 R_1 R_0 = R_{\text{ext}}. \quad (14)$$

Thus fixed-axis rotations pre-multiply in the listed order, whereas body-axis rotations post-multiply in the reverse order, producing the same final matrix.

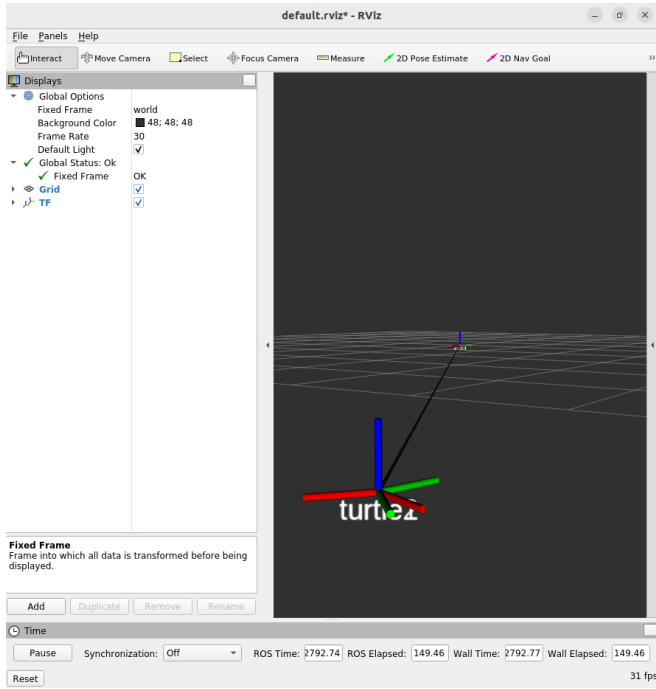


Fig. 24. rViz with the TF display enabled and the world frame selected as fixed frame.

For the numerical sequence in the handout,

$$R_0 = R_x(90^\circ), \quad R_1 = R_y(180^\circ), \quad R_2 = R_x(-30^\circ). \quad (15)$$

The common final orientation is

$$R_2 R_1 R_0 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1/2 & -\sqrt{3}/2 \\ 0 & -\sqrt{3}/2 & 1/2 \end{bmatrix}. \quad (16)$$

The optional deliverable is therefore completed analytically; it does not require an additional runtime experiment.

V. RESULTS

The actual results are summarized in Table II. The table distinguishes compilation, visualization, and mathematical verification; no unsupported physical-robot result is included.

The evidence also records the package-version issue. The current repository tip was not silently treated as ROS1: it was identified as an `ament_cmake` package and was skipped by catkin. The successful result was obtained only after selecting the historical ROS1 commit 609f31f. This distinction is important because a superficially successful catkin command can still mean that a package was ignored.

VI. EVIDENCE AND VERIFICATION

Table III establishes the required requirement–operation–result–evidence mapping. The figure paths are relative to the report’s `figures/` directory, and the source paths are relative to the report’s `code/` directory.

The screenshots are evidence rather than decoration. The Docker screenshots establish host/container setup, the terminal screenshots establish commands and build status, the

TABLE II
VERIFIED LAB2 RESULTS.

Check	Status	Recorded result
Docker daemon access	Passed	<code>docker ps</code> ran without permission error
ROS environment	Passed	<code>rosversion -d:</code> <code>noetic</code>
Workspace initialization	Passed	Catkin configuration reported valid
ROS1 package build	Passed	One package succeeded; none ignored or failed
Static rViz scene	Passed	Two drone models and global status OK
Node/topic inspection	Passed	Nodes and TF topics listed; rqt graph displayed
Dynamic scene	Passed	Two time-separated rViz views showed changed positions
Relative-frame view	Passed	Fixed frame <code>av1</code> ; circle, parabola, and dashed relative path visible
Turtlesim teleoperation	Passed	Turtlesim and the keyboard teleoperation endpoint launched
Twist motion controller	Passed	A custom ROS node published 12 s of velocity commands and then stopped
Intrinsic/extrinsic rotation proof	Passed	Matrix products agreed; final orientation given by Eq. (15)
Physical execution	Not performed	No physical UAV or robot was connected

rViz screenshots establish visualization output, and the rqt graph establishes the ROS communication structure. The screenshots do not prove a physical robot experiment, sensor calibration, timing accuracy, or hardware safety; those items were outside the recorded scope.

VII. DISCUSSION

The results agree with the expected behavior. The static launch produced the two vehicle models and a valid TF visualization. The dynamic launch produced time-separated positions, and changing the fixed frame to AV1 revealed the predicted geometric relationship. The dashed relative path in Fig. 15 is consistent with the derived planar ellipse.

The most important implementation detail was the orientation of AV1. A translation-only implementation could still show a circular AV1 path and a moving AV2, but it would produce an incorrect relative trajectory because the body frame would not rotate with the vehicle. The use of yaw t makes the body y -axis tangent to the circular path and is therefore necessary for the plane and ellipse derivations.

The most important environment issue was the mismatch between the current repository tip and the older ROS1 exercise. The first catkin invocation reported a successful command overall but explicitly skipped the package because its build type was `ament_cmake`. Treating that output as a real package build would have produced an incorrect completion claim. Full history inspection and the switch to 609f31f resolved the issue.

TABLE III
REQUIREMENT, OPERATION, RESULT, AND EVIDENCE MAPPING.

Requirement	Operation	Verified result	Evidence
Install usable ROS environment	Install Docker, pull Noetic image, source ROS	Noetic container available on Ubuntu 22.04 host	Figs. 1–3
Create and build workspace	Initialize catkin and build ROS1 package	One package succeeded; no ignored or failed packages	Figs. 5, 7
Inspect ROS graph	Run <code>roscpp list</code> , <code>rostopic list</code> , and <code>rqt_graph</code>	Broadcasters, plotting node, TF topics, and graph edges observed	Figs. 9 and 10
Publish transforms	Complete the 50 Hz publisher and rebuild	<code>world->av1</code> and <code>world->av2</code> visualized	Fig. 11; code attachment
Query relative transform	Complete TF2 lookup and rebuild	Relative path visible in AV1 frame	Figs. 14 and 15; code attachment
Verify ROS mobile-robot control	Launch turtlesim and teleoperation	Turtlesim and teleoperation endpoint launched	Fig. 16
Publish motion commands	Run the custom Twist controller	Circular trace and stop message recorded	Fig. 17; controller code
Analyze geometry	Derive position, plane, ellipse, and quaternion properties	Equations (1)–(7) and quaternion identities	Sec. IV
Prove rotation equivalence	Reverse intrinsic order and compare matrix products	Equations (13)–(15) agree	Sec. IV
Update rosdep cache	Run <code>rosdep update</code> after DNS repair	Source cache updated successfully	Fig. 18
Inspect TurtleSim topic graph	Echo <code>/turtle1/cmd_vel</code> and run <code>rqt_graph</code>	Velocity messages and publisher/subscriber graph observed	Figs. 19 and 20
Inspect TF tools	Run turtle TF demo, <code>rqt_tf_tree</code> , <code>tf_echo</code> , and <code>rViz TF</code>	Frame tree, numerical transform, and TF display verified	Figs. 21–24

The additional turtlesim tests completed the previously missing command-to-motion portion of the handout. They verify a ROS mobile-robot simulation and the `Twist` interface, but they do not constitute a physical UAV test. The controller’s fixed duration and constant velocities make the result easy to repeat, while the absence of a recorded quantitative odometry comparison limits the claim to successful visible motion and command completion.

The experiment is reproducible at the software-visualization level using the recorded host/container arrangement, the ROS1 commit, the workspace commands, and the included screenshots. Exact reproduction of the `rViz` camera view may vary because the screenshots record interactive camera state, and the report does not include the complete original container filesystem. No physical hardware test, quantitative timing benchmark, or independent numerical comparison against logged trajectory data was recorded. Those are limitations, not missing claims to be filled by assumption.

VIII. PERSONAL REFLECTION AND ANALYSIS

The main lesson was that an experiment is a chain of verified states rather than a sequence of commands. The Docker installation was not considered complete until the daemon was reachable without a permission error. The ROS installation was not considered complete until the container returned `noetic`. The first catkin command was not considered a package build because the package was skipped, even though catkin itself returned a summary containing no failed jobs.

The turtlesim extension made the ROS communication path concrete: a keyboard node and a custom controller both publish to a velocity topic, while the simulator turns those messages into motion. This separated the generic ROS

command-to-motion workflow from the more specialized TF trajectory exercise. It also showed why a running ROS master and a running consumer node must be checked separately; a controller can publish successfully while no visible motion occurs if the simulator has already been closed.

The code exercise clarified the relationship between motion equations and coordinate-frame conventions. The formula $[\cos t, \sin t, 0]^T$ defines only the origin trajectory; the orientation determines how the same physical motion is represented from the vehicle frame. The TF lookup direction is similarly not a naming detail. Swapping the target and source frames would change the meaning of the returned transform and could produce a visually plausible but mathematically different curve.

The most transferable practice was preserving diagnostic evidence. The skipped ROS2 build was not deleted from the evidence set, because it explains why a historical checkout was required. In a later robotics project, the same approach can prevent a build log, launch file, or installed package from being mistaken for proof that the intended code actually ran.

If the experiment were repeated, I would add an automated numerical check that samples $o_2^1(t)$ and evaluates the residuals of $z - 2y - 1$ and the ellipse equation. I would also record the complete container image digest, the exact package tree, and a clean rebuild from an exported archive. These additions would strengthen reproducibility beyond the current visual and compile evidence.

IX. CONCLUSIONS

The Lab2 objectives were achieved for the recorded software-in-the-loop scope. A ROS1 Noetic container was established on the available Ubuntu 22.04 virtual machine, a ROS1-compatible package version was selected, and the

catkin workspace compiled successfully. The turtlesim teleoperation and Twist-controller tests completed the command-to-motion exercise. The static and dynamic two-drone scenarios ran in rViz, the ROS graph was inspected, the TF publisher and relative-transform lookup were implemented, and the AV1-fixed visualization displayed the expected trajectories.

The mathematical analysis confirmed that AV2 follows a world-frame parabola, that its AV1-relative trajectory lies on $z - 2y = 1$, and that the centered planar trajectory is an ellipse with semi-axis lengths $1/2$ and $\sqrt{5}/2$. The final evidence supports compilation and simulated visualization. It does not claim physical-robot execution, hardware validation, or quantitative timing accuracy, because those were not recorded.

REPRODUCIBILITY STATEMENT

The following items were reproduced and recorded: Docker daemon access, the ROS Noetic version check, catkin workspace initialization, package compilation at commit 609f31f, turtlesim teleoperation, custom Twist-based

motion, static and dynamic rViz visualization, node/topic inspection, rqt graph inspection, the AV1-fixed relative trajectory, and the optional intrinsic/extrinsic rotation proof. The mathematical derivations were independently written from the recorded motion equations. Physical execution, hardware timing, odometry-error benchmarking, and a clean rebuild from a newly exported container were not verified and require further testing.

REFERENCES

- [1] Y. Yang, "LAB2: ROS Installation and Usage," course handout, 2025. [Attached PDF.]
- [2] MIT VNAV, "Exercises: Lab 2," 2023. [Online]. Available: https://vnav.mit.edu/labs_2023/lab2/exercises.html
- [3] MIT-SPARK, "VNAV-labs," GitHub repository. [Online]. Available: <https://github.com/MIT-SPARK/VNAV-labs>
- [4] Open Robotics, "ROS Noetic Ninjemys," ROS documentation. [Online]. Available: <https://wiki.ros.org/noetic>
- [5] Open Robotics, "Writing a tf2 listener (C++)," ROS documentation. [Online]. Available: <https://wiki.ros.org/tf2/Tutorials/Writing%20a%20tf2%20listener%20%28C%2B%2B%29>
- [6] Open Source Robotics Foundation, "osrf/ros Docker images," Docker Hub. [Online]. Available: <https://hub.docker.com/r/osrf/ros/tags?name=noetic&page=1>